

TP 2 : Construire un mini RAG avec LangChain, FAISS, PromptTemplate et Groq

Module : IA Générative

TP guidé de l'installation jusqu'au test sur des PDF

Nom et prénom		Groupe	
Date		Durée	2h00

1. Objectifs du TP

À la fin de ce TP, l'étudiant doit être capable de :

- expliquer les deux phases d'un système RAG : indexation et question-réponse ;
- charger des documents PDF en Python ;
- découper les documents en *chunks* ;
- créer des embeddings et construire un index **FAISS** ;
- utiliser **LangChain** et **PromptTemplate** pour formater une requête ;
- interroger un modèle **Groq** depuis Python ;
- tester et analyser la qualité d'un mini chatbot RAG.

2. Idée générale du TP

Dans ce TP, on veut construire une application simple :

1. lire plusieurs fichiers PDF ;
2. extraire leur texte ;
3. les découper en morceaux ;
4. transformer ces morceaux en vecteurs ;
5. stocker ces vecteurs dans FAISS ;
6. poser une question ;
7. retrouver les extraits les plus pertinents ;
8. envoyer ces extraits à un modèle Groq avec un **PromptTemplate** ;
9. obtenir une réponse fondée sur les documents.

3. Rappel théorique très court

RAG signifie *Retrieval-Augmented Generation*.

Indexation :

- charger les documents ;
- les découper en *chunks* ;
- calculer les embeddings ;
- stocker les vecteurs.

Question-réponse :

- transformer la question en requête ;
- récupérer les *top-k* chunks les plus proches ;

- construire un prompt avec ce contexte ;
- demander au LLM de répondre.

4. Outils utilisés

Dans ce TP, on utilise :

- **Python** pour le code ;
- **LangChain** pour organiser les briques ;
- **PyPDFLoader** pour lire les PDF ;
- **RecursiveCharacterTextSplitter** pour le chunking ;
- **HuggingFaceEmbeddings** pour calculer les vecteurs ;
- **FAISS** pour l'index vectoriel ;
- **PromptTemplate** pour formater proprement la consigne ;
- **ChatGroq** pour interroger le modèle.

5. Avant de commencer

Pré-requis :

- Python 3.10 ou 3.11 ;
- un terminal ;
- une connexion Internet ;
- des PDF à tester.

Remarque importante : sur certaines machines Windows, `faiss-cpu` peut demander un environnement Python propre. Si besoin, utiliser un environnement virtuel neuf.

6. Comment obtenir une clé API Groq ?

1. ouvrir <https://console.groq.com> ;
2. créer un compte ou se connecter ;
3. ouvrir la zone **API Keys** ;
4. cliquer sur **Create API Key** ;
5. copier la clé générée ;
6. la stocker dans un fichier `.env`.

Contenu du fichier `.env` :

```
1 GROQ_API_KEY=votre_cle_api_ici
```

Bonne pratique : ne jamais écrire la clé directement dans le code Python et ne jamais la partager.

7. Organisation du projet

Créer un dossier de travail avec la structure suivante :

```
1 tp_rag/  
2 |-- data/  
3 | |-- chapitre_rag.pdf  
4 | |-- chapitre_prompt_engineering.pdf
```

```

5 | |-- autres_documents.pdf
6 |-- .env
7 |-- rag_pdf_groq.py

```

Important : le dossier `data/` doit contenir les PDF à indexer et à interroger.

Conseil pour ce TP : utiliser au moins deux PDF du module, par exemple :

- `chapitre_rag.pdf`
- `chapitre_prompt_engineering.pdf`

8. Installation des dépendances

Sous Windows :

```

1 python -m venv .env
2 .env\Scripts\activate
3 pip install --upgrade pip
4 pip install langchain langchain-core langchain-community langchain-text-splitters
5 pip install langchain-huggingface langchain-groq
6 pip install faiss-cpu sentence-transformers pypdf python-dotenv

```

Sous Linux / macOS :

```

1 python3 -m venv .env
2 source .env/bin/activate
3 pip install --upgrade pip
4 pip install langchain langchain-core langchain-community langchain-text-splitters
5 pip install langchain-huggingface langchain-groq
6 pip install faiss-cpu sentence-transformers pypdf python-dotenv

```

9. Test rapide de la clé Groq

Avant de lancer le vrai TP, il faut vérifier que la clé est reconnue.

```

1 python -c "from dotenv import load_dotenv; import os; load_dotenv(); print(os.getenv('GROQ_API_KEY') is not None)"

```

Résultat attendu :

- `True` si la clé est bien chargée;
- `False` si le fichier `.env` est absent ou mal rempli.

10. Code complet du TP

Enregistrer le code Python dans un fichier nommé `rag_pdf_groq.py`.

Listing 1 – Code complet du mini RAG sur PDF avec LangChain, FAISS et Groq

```

1 import os
2 from pathlib import Path
3
4 from dotenv import load_dotenv
5 from langchain_community.document_loaders import PyPDFLoader
6 from langchain_community.vectorstores import FAISS
7 from langchain_core.prompts import PromptTemplate
8 from langchain_groq import ChatGroq
9 from langchain_huggingface import HuggingFaceEmbeddings
10 from langchain_text_splitters import RecursiveCharacterTextSplitter
11
12
13 DATA_DIR = Path("data")
14 EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
15 GROQ_MODEL = "llama-3.3-70b-versatile"
16 CHUNK_SIZE = 800
17 CHUNK_OVERLAP = 150
18 TOP_K = 3

```

```
19
20
21 def load_pdf_documents(data_dir: Path):
22     pdf_paths = sorted(data_dir.glob("*.pdf"))
23
24     if not pdf_paths:
25         raise FileNotFoundError(
26             "Aucun PDF trouve dans le dossier 'data'. "
27             "Ajoutez par exemple chapitre_rag.pdf et chapitre_prompt_engineering.pdf."
28         )
29
30     documents = []
31
32     for pdf_path in pdf_paths:
33         loader = PyPDFLoader(str(pdf_path))
34         pages = loader.load()
35
36         for page in pages:
37             page.metadata["source"] = pdf_path.name
38
39         documents.extend(pages)
40
41     return documents
42
43
44 def split_documents(documents):
45     splitter = RecursiveCharacterTextSplitter(
46         chunk_size=CHUNK_SIZE,
47         chunk_overlap=CHUNK_OVERLAP,
48         separators=["\n\n", "\n", ". ", " ", ""],
49     )
50     return splitter.split_documents(documents)
51
52
53 def create_vectorstore(chunks):
54     embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
55     return FAISS.from_documents(chunks, embeddings)
56
57
58 def build_prompt():
59     template = """
60 Tu es un assistant pedagogique specialise en IA generative.
61 Tu dois repondre uniquement a partir du contexte fourni.
62
63 Consignes importantes :
64 - Reponds en francais.
65 - Si l'information n'est pas presente dans le contexte, dis clairement :
66   "Je ne trouve pas cette information dans les documents fournis."
67 - Donne une reponse claire, structuree et concise.
68 - Termine par une ligne "Sources :" avec les fichiers utilises.
69
70 Contexte :
71 {context}
72
73 Question :
74 {question}
75 """
76     return PromptTemplate.from_template(template)
77
78
79 def format_context(docs):
80     parts = []
81
82     for i, doc in enumerate(docs, start=1):
83         source = doc.metadata.get("source", "source_inconnue")
84         page = doc.metadata.get("page", "?")
85         if isinstance(page, int):
86             page = page + 1
87
88         parts.append(
89             f"[Extrait {i} | source={source} | page={page}]\n{doc.page_content}"
90         )
91
92     return "\n\n".join(parts)
93
```

```
94
95 def format_sources(docs):
96     unique_sources = []
97
98     for doc in docs:
99         source = doc.metadata.get("source", "source_inconnue")
100         page = doc.metadata.get("page", "?")
101         if isinstance(page, int):
102             page = page + 1
103
104         item = f"{source} (page {page})"
105         if item not in unique_sources:
106             unique_sources.append(item)
107
108     return ", ".join(unique_sources)
109
110
111 def answer_question(question, retriever, llm, prompt):
112     docs = retriever.invoke(question)
113     context = format_context(docs)
114     sources = format_sources(docs)
115
116     final_prompt = prompt.format(context=context, question=question)
117     response = llm.invoke(final_prompt).content
118
119     if "Sources :" not in response:
120         response = response.strip() + f"\n\nSources : {sources}"
121
122     return response, docs
123
124
125 def main():
126     load_dotenv()
127
128     groq_api_key = os.getenv("GROQ_API_KEY")
129     if not groq_api_key:
130         raise EnvironmentError(
131             "La variable GROQ_API_KEY est absente. "
132             "Ajoutez-la dans un fichier .env ou dans vos variables d'environnement."
133         )
134
135     print("Chargement des PDF...")
136     documents = load_pdf_documents(DATA_DIR)
137     print(f"Nombre total de pages chargees : {len(documents)}")
138
139     print("Decoupage en chunks...")
140     chunks = split_documents(documents)
141     print(f"Nombre total de chunks : {len(chunks)}")
142
143     print("Creation des embeddings et de l'index FAISS...")
144     vectorstore = create_vectorstore(chunks)
145     retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})
146
147     llm = ChatGroq(
148         model=GROQ_MODEL,
149         temperature=0,
150         api_key=groq_api_key,
151     )
152     prompt = build_prompt()
153
154     print("\nAssistant RAG pret.")
155     print("Tapez votre question ou 'quit' pour quitter.\n")
156
157     while True:
158         question = input("Question > ").strip()
159
160         if not question:
161             print("Veuillez saisir une question.\n")
162             continue
163
164         if question.lower() in {"quit", "exit", "q"}:
165             print("Fin du programme.")
166             break
167
168         answer, docs = answer_question(question, retriever, llm, prompt)
```

```
169     print("\n--- Reponse ---")
170     print(answer)
171
172     print("\n--- Chunks recuperes ---")
173     for i, doc in enumerate(docs, start=1):
174         source = doc.metadata.get("source", "source_inconnue")
175         page = doc.metadata.get("page", "?")
176         if isinstance(page, int):
177             page = page + 1
178
179         preview = doc.page_content[:250].replace("\n", " ")
180         print(f"{i}. {source} | page {page}")
181         print(f" {preview}...")
182
183     print()
184
185
186
187 if __name__ == "__main__":
188     main()
```

11. Explication du code pas à pas

11.1. Chargement des bibliothèques

Le script importe :

- `load_dotenv` pour lire la clé Groq ;
- `PyPDFLoader` pour ouvrir les PDF ;
- `RecursiveCharacterTextSplitter` pour faire le chunking ;
- `HuggingFaceEmbeddings` pour les vecteurs ;
- `FAISS` pour la recherche vectorielle ;
- `PromptTemplate` pour construire le prompt ;
- `ChatGroq` pour générer la réponse finale.

11.2. Paramètres importants

Le script fixe dès le début :

- le dossier des PDF ;
- le modèle d'embedding ;
- le modèle Groq ;
- la taille des chunks ;
- l'overlap ;
- la valeur `TOP_K`.

Interprétation :

- `CHUNK_SIZE = 800` : chaque morceau contient environ 800 caractères ;
- `CHUNK_OVERLAP = 150` : deux morceaux se recouvrent partiellement ;
- `TOP_K = 3` : on récupère 3 extraits pertinents.

11.3. Chargement des PDF

La fonction `load_pdf_documents()` :

- cherche tous les PDF dans `data/` ;
- lit chaque page ;

- ajoute le nom du fichier dans les métadonnées ;
- fusionne toutes les pages dans une seule liste de documents.

Pourquoi les métadonnées sont utiles ? Parce qu'elles permettent d'afficher la source du passage retrouvé.

11.4. Chunking

La fonction `split_documents()` découpe les pages en morceaux plus petits.

Pourquoi ne pas garder le PDF entier ?

- un document entier est trop long ;
- la recherche est moins précise ;
- un petit extrait est plus facile à injecter dans le prompt.

11.5. Embeddings et FAISS

La fonction `create_vectorstore()` :

- transforme chaque chunk en vecteur ;
- stocke tous les vecteurs dans FAISS.

Idée essentielle : au moment d'une question, on ne compare pas directement des phrases, on compare des vecteurs.

11.6. PromptTemplate

Le `PromptTemplate` contient :

- une identité d'assistant ;
- une consigne de prudence ;
- le contexte ;
- la question.

Bonne pratique du TP : le prompt demande explicitement au modèle de dire qu'il ne sait pas si l'information n'est pas dans les documents.

11.7. ChatGroq

`ChatGroq` sert à appeler le modèle de génération.

Rôle de Groq ici :

- recevoir le contexte ;
- lire la question ;
- produire une réponse finale claire.

11.8. Boucle interactive

La fonction `main()` :

- charge la clé API ;
- charge les PDF ;
- crée les chunks ;
- crée l'index FAISS ;
- initialise le modèle Groq ;
- attend les questions de l'utilisateur.

Intérêt pédagogique : les étudiants voient clairement les deux grandes phases du RAG dans un vrai programme.

12. Exécution du programme

Lancer le script avec :

```
1 python rag_pdf_groq.py
```

Exemples de questions à tester :

- Qu'est-ce que le RAG ?
- Quel est le rôle de FAISS ?
- À quoi sert PromptTemplate ?
- Quelle différence entre zero-shot et few-shot ?

13. Activités pratiques à réaliser

Activité 1 : premier test

Lancer le programme et poser deux questions qui sont clairement présentes dans vos PDF.

Question 1 :

Réponse obtenue :

Question 2 :

Réponse obtenue :

Activité 2 : question absente du corpus

Poser une question qui n'est pas dans les PDF, par exemple sur un sujet totalement différent.

Question testée :

Comportement observé :

Conclusion : le prompt empêche-t-il bien le modèle d'inventer ?

Activité 3 : modifier le chunking

Changer les paramètres suivants dans le code :

- `CHUNK_SIZE = 400` puis `CHUNK_SIZE = 1000`
- `CHUNK_OVERLAP = 50` puis `CHUNK_OVERLAP = 200`

Observation : quel réglage semble le plus efficace sur vos PDF ?

Activité 4 : modifier le nombre de chunks retrouvés

Changer TOP_K :

- TOP_K = 1
- TOP_K = 3
- TOP_K = 5

Question testée :

Effet sur la qualité de la réponse :

Activité 5 : modifier le prompt

Ajouter une contrainte dans le `PromptTemplate`, par exemple :

- répondre en 5 points ;
- donner une réponse très courte ;
- citer systématiquement les sources.

Prompt modifié :

Effet observé :

Activité 6 : ajouter un nouveau PDF

Ajouter un nouveau document dans `data/`, relancer le programme puis poser une question qui dépend de ce nouveau document.

Nom du PDF ajouté :

Question testée :

Conclusion : le système a-t-il bien pris en compte le nouveau document ?

14. Questions d'analyse

1. Pourquoi faut-il découper les documents en chunks ?
2. Pourquoi utilise-t-on des embeddings ?
3. Quel est le rôle exact de FAISS ?
4. Quelle est la différence entre le retriever et le LLM ?
5. Pourquoi un `PromptTemplate` bien écrit améliore-t-il le RAG ?
6. Que se passe-t-il si la clé `GROQ_API_KEY` est absente ?
7. Quel paramètre vous semble le plus important : chunk size, overlap ou top-k ?

15. Livrable final

Le livrable final doit contenir :

- la liste des PDF utilisés ;
- une capture du dossier `data/` ;
- au moins 3 questions posées au système ;
- les réponses obtenues ;
- un test avec une question absente du corpus ;
- un test comparatif avec deux réglages différents de chunking ou de `TOP_K` ;
- une conclusion personnelle de 8 à 12 lignes.

16. Grille d'évaluation

Critère	Barème	Note
Installation correcte de l'environnement et usage de Groq	4 pts	
Compréhension de la chaîne RAG complète	4 pts	
Qualité des tests sur les PDF et des observations	4 pts	
Capacité à expliquer chunking, embeddings, FAISS et prompt	4 pts	
Qualité du livrable final et clarté de la conclusion	4 pts	
Total	20 pts	

17. Ressources utiles

- Groq Console : <https://console.groq.com>
- Groq API Docs : <https://console.groq.com/docs>
- LangChain : <https://python.langchain.com>
- FAISS : <https://faiss.ai>

Idée à retenir : un bon RAG ne dépend pas seulement du modèle, mais aussi de la qualité des documents, du chunking, de la recherche et du prompt.